I build and run production AI automation for a live, money-critical business.

Statement pipelines, a WhatsApp assistant and finance tools in use at Trusted Figures, a London accountancy firm.

Looking for an automation, integration or solutions engineering role in London, starting February 2027.

ladopaichadze@gmail.com / LinkedIn

01 / 05

CIS statement pipeline

- Live: intake, matching, filing and client dashboards; chasing in first use

41	client dashboards at launch, Aug 2026[1]
236	expected statements tracked in one quarter 38 clients, Q1 2026-27, Oct 2026[2]

Staff see which contractor statements each client still owes, without checking by hand.

02 / 05

AI chatbot on WhatsApp and the website, with a team dashboard

- Live; one firm in production

42 of 42	acceptance benchmark cases passed live AI model, Jul 2026[3]

Clients get answers on WhatsApp and the website; staff step in from one dashboard.

03 / 05

Making Tax Digital figures from bank statements

- In internal use for quarterly Making Tax Digital submissions

140 of 140	real statements self-verified 14,801 transactions, 0 parse failures, Jul 2026[4]
9	bank statement layouts read 11 UK bank brands, 10 tested on real statements, Aug 2026[5]

Turns a sole trader's bank statements into checked figures for their quarterly tax update.

04 / 05

Business automations in n8n

- Live

13	active workflows live n8n instance, Oct 2026[6]

Client messages, documents, registrations and filing deadlines are handled in the background; staff are told what needs them.

05 / 05

Making an AI-assisted codebase follow its own rules

- Live; remaining gaps signed off by name

over 500
prose rules for AI coding sessions
before the review, Oct 2026[7]

8
rules today
must-never rules, plus one page of advice (40 items), Oct 2026[8]

9
required checks
every merge to main, Oct 2026[9]

Every change an AI coding assistant makes must pass the same required checks before it reaches the live code.

01 of 05

CIS statement pipeline

Staff see which contractor statements each client still owes, without checking by hand.

Status	Stack	Role
Live: intake, matching, filing and client dashboards; chasing in first use	Python, PostgreSQL, Self-hosted OCR, OpenAI, n8n	Designed and built

41
client dashboards
at launch, Aug 2026[1]

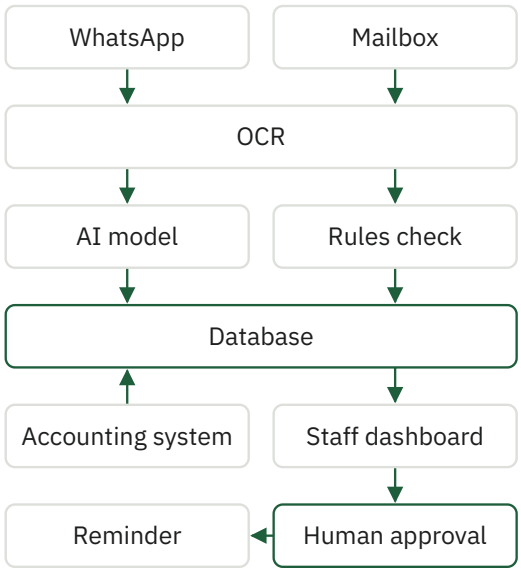
236
expected statements tracked in one quarter
38 clients, Q1 2026-27, Oct 2026[2]

Problem

Construction subcontractors have tax deducted by every contractor they work for, and each deduction needs a statement from that contractor before it can be claimed back. Collecting them by photo and email, and tracking which are still missing, is slow work by hand.

What I built

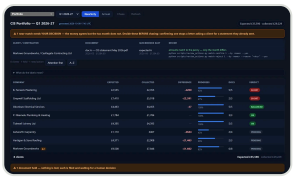
A pipeline that starts each quarter from the accounting system's list of every deduction the firm expects. Each statement a client sends by WhatsApp or email is read, identified and ticked off against that list, and staff see what is missing for every client. For what is missing it prepares chase messages, which a person approves and sends.

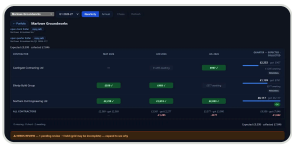


Under the hood

- Filed only when three signals agree: who sent it, the company it names, and the contractor and month. Anything unclear waits for a person.

- OCR runs on our own machine; no page image goes to a cloud OCR service. An AI model and an independent rules check both read the extracted text; any disagreement means hold.
- Files are fingerprinted, so a re-sent statement counts once. The mailbox sweep has run unattended every 15 minutes since 20 Aug 2026.
- Python and PostgreSQL, money as exact decimals, **3,807**[10] automated tests.





Results

- **54**[11] statements matched for Q1 2026-27 and filed with no hand checking.
- **15**[12] exact duplicate files caught and counted once.

Decisions and trade-offs

- Held, never guessed: three signals must agree before a statement is filed. More documents wait for a person, but no wrong match reaches a client file.
- Chasing is attended: a person confirms each send, one client at a time, in a step separate from composing. It costs staff time, but a wrong or repeated message cannot reach a client.

02 of 05

AI chatbot on WhatsApp and the website, with a team dashboard

Clients get answers on WhatsApp and the website; staff step in from one dashboard.

Status	Stack	Role
Live; one firm in production	TypeScript, Supabase, n8n, WhatsApp Cloud API, OpenAI, Anthropic	Designed and built

42 of 42

acceptance benchmark cases passed

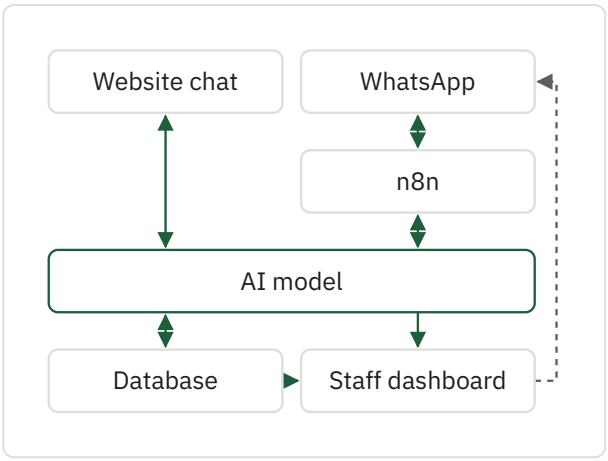
live AI model, Jul 2026[3]

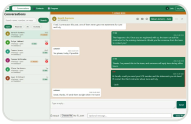
Problem

Clients and would-be clients would rather ask a question or send a photo on WhatsApp than write an email. A growing office cannot answer them all quickly and consistently without pulling staff away from the work that matters more.

What I built

An AI chatbot that answers clients on WhatsApp and on the firm's website, in 6[13] languages. It holds the conversation for 30 minutes and keeps a short summary of earlier ones, plus facts a client confirms such as their name and language, so returning clients need not repeat themselves. When a person is needed, it hands over to staff, who read every thread and reply under their own name from one team dashboard.









Under the hood

- Common personal identifiers are masked before the AI sees a message.
- Answers come only from the firm's approved service material, and fixed safety code checks each reply before it is sent.
- Each AI call has a timeout and a retry, then switches to a second AI provider.
- Built multi-tenant: every record carries a tenant key and every query filters on it.
- TypeScript on Supabase, with n8n carrying every WhatsApp message in and out; 1,315[14] automated tests.

Results

- Statements clients send on WhatsApp go straight into the CIS statement pipeline, which reads, sorts, renames and files them.
- Website visitors asking about CIS or a UTR (tax reference) register inside the chat; staff are alerted on WhatsApp.
- Admins set, confirm and cancel reminders by plain WhatsApp message.

Decisions and trade-offs

- Hand over, never guess: when a person is needed, the bot asks one short question and passes the thread to staff. It costs staff time, but a client never gets a guessed answer about their tax.
- Unattended messages are pre-approved templates only, each checked for duplicates, opt-outs and the 24-hour messaging window, and a blocked send is logged with its reason. Reminders are less flexible, but no free text reaches a client without a person.
- New features ship switched off and are turned on deliberately. Rollout is slower, but a half-finished feature never reaches a client.

Debugging story

A client sent eight photos at once and only some were stored. Each new file was first saved under the same placeholder name, which the database accepts only once, so photos arriving together were rejected. Each file now gets its own placeholder; a test reproduces the loss, and a live burst test after the fix stored **8 of 8**[15] photos, each paired to its message.

03 of 05

Making Tax Digital figures from bank statements

Turns a sole trader's bank statements into checked figures for their quarterly tax update.

Status	Stack	Role
In internal use for quarterly Making Tax Digital submissions	Python, PDF text parsing, Rules engine	Designed and built

140 of 140
real statements self-verified
14,801 transactions, 0 parse failures, Jul 2026[4]

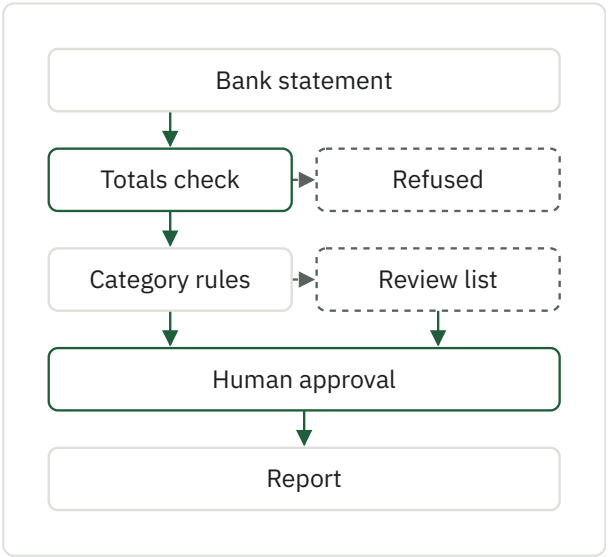
9
bank statement layouts read
11 UK bank brands, 10 tested on real statements, Aug 2026[5]

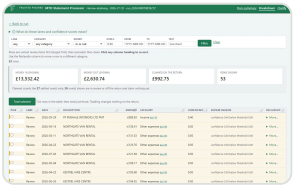
Problem

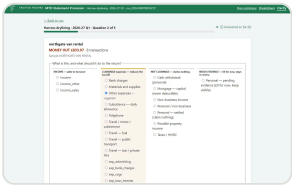
Every quarterly tax update starts from a pile of bank statements, and each line has to be read, sorted into income or expense and totalled. By hand it takes so long that a busy office turns new clients away, and one misread or missorted line changes the figure that gets filed.

What I built

A tool that reads each statement, re-adds the lines to the bank's printed totals and running balance, and refuses one that does not add up. Fixed rules sort every line into a category, and anything uncertain goes to a review list instead of the figures. A step by step screen asks about unknown merchants one at a time and shows each answer's effect on the return before saving it. A person always reviews and files; the tool sends nothing to the tax authority.







Under the hood

- Processing is local; the tool makes no network calls. Reading and sorting use fixed rules, no AI, and an automated check refuses code that would add either.

- Rules sit in three layers: the client's rules beat the trade's, and the trade's beat the firm's. One client can switch off a trade rule without touching anyone else.
- Money is held as exact decimals, never floating point, and statement files are read only to the tool.
- Python, with lint, strict type checks and **976**[16] automated tests required on every change.

Results

- A rule change meant for one client would have moved a figure on another client's filed return. The check on filed figures stopped it before it landed.
- Cash taken out at a supermarket cashpoint was about to count as a claimable meal cost. An automated test caught it before any return was filed.

Decisions and trade-offs

- Fixed rules, no AI: the same statements and rules always give the same figures, and every category traces back to a rule. A new client means more questions for the accountant at first, but no figure moves without a reason.
- Read the text the PDF already holds, not a picture of the page. Scanned statements are refused for now, but every amount is taken as the bank printed it, and the totals check proves it.
- Local and single user: the page answers only on the same computer and has no login, so one person uses it today. A hosted version for more staff waits until it has proved itself on real quarters.

04 of 05

Business automations in n8n

Client messages, documents, registrations and filing deadlines are handled in the background; staff are told what needs them.

Status	Stack	Role
Live	n8n, WhatsApp Cloud API, Microsoft Graph, Companies House API, OpenAI embeddings, Supabase	Designed and built

13
active workflows
live n8n instance, Oct 2026[6]

Problem

Client work arrives on WhatsApp and through a website form, and every client company has filing deadlines to watch. By hand it is slow, and a dropped step or a missed deadline goes unnoticed until it is late.

What I built

Background workflows in n8n. The **13**[17] active workflows, by job:

Messages in

`2 workflows`

WhatsApp text goes to the chatbot, files to the document pipeline; an older email route runs only when started by hand.

Messages out

`2 workflows`

Reminders and template messages to clients leave through two guarded send workflows, never straight from an app.

Document filing

`3 workflows`

Each read document is filed in the right client folder in cloud storage; two helpers clear stale copies and empty folders.

Registrations

`1 workflow`

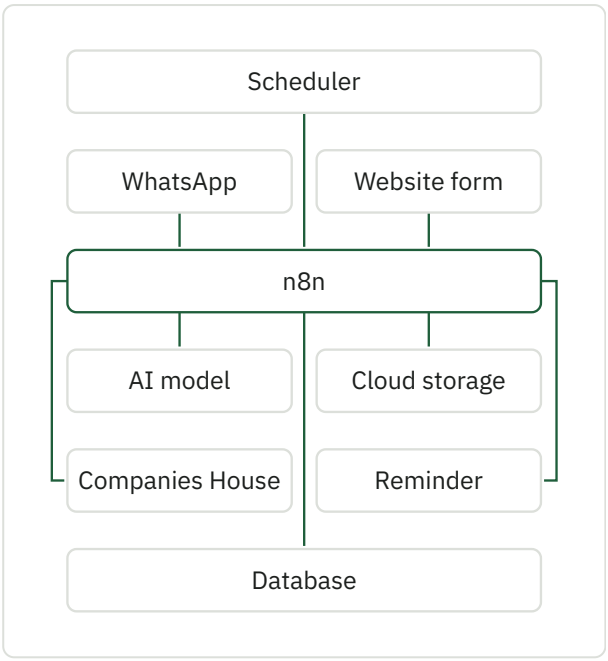
Each website registration gets a client folder with its ID photos and a summary, and staff are told on WhatsApp.

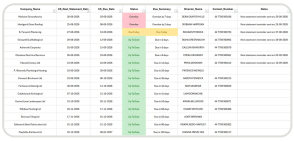
Compliance

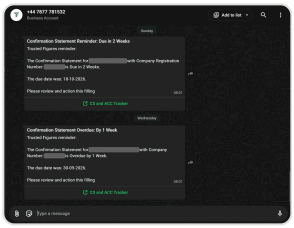
`5 workflows`

Two trackers check every company at Companies House each morning and remind staff; one feeds the staff dashboard; two refresh on request.

Also built, now switched off: the chatbot's first version, a similarity search over FAQs and HMRC guidance. It never answered a client; it still runs when started by hand.







Under the hood

- Every call from the firm's own apps into a workflow needs a shared secret or token; no credentials sit in the workflow files.
- Reminder sends refuse duplicates with a one-time key, reserved before sending, and retry automatically.
- Repeat WhatsApp deliveries are dropped on the provider's message id, and failures alert staff.
- Incoming files are checked against allowed types, and a sender who sends several files gets one combined reply.
- Only approved templates go out and blank fields are refused; filing retries before alerting staff, and a folder is deleted only when it is empty.

Results

- Staff never work in n8n or file these items by hand; they see the results on their dashboards and in WhatsApp alerts.

Decisions and trade-offs

- Client reminders and templates leave through two send workflows, not from each app. It adds a hop, but every client message passes the same checks.
- A failed WhatsApp reply cannot block a document from being saved: the two run separately. A client may wait for an answer, but no file is lost.
- The live workflows are exported to git, because n8n keeps no edit history of its own. The exports are only as current as the last export, but a broken edit can be compared with the last good copy.

05 of 05

Making an AI-assisted codebase follow its own rules

Every change an AI coding assistant makes must pass the same required checks before it reaches the live code.

Status	Stack	Role
Live; remaining gaps signed off by name	GitHub Actions, gitleaks, Claude Code	Designed and built

over 500
prose rules for AI coding sessions
before the review, Oct 2026[7]

8
rules today
must-never rules, plus one page of advice (40 items), Oct 2026[8]

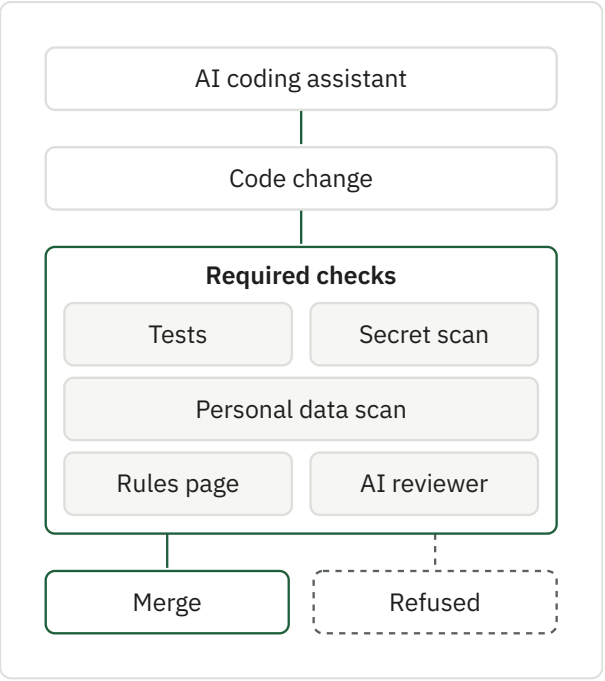
9
required checks
every merge to main, Oct 2026[9]

Problem

When AI coding assistants write much of the code, the rules for how they should work pile up as prose that nothing checks. Local checks can be skipped, and an assistant can call a job done without proof.

What I built

A review that moved enforcement off the laptop and onto the server: the main branch is protected, and a change merges only when every required check passes. The rules became a short list that must never break, each refused by an automated check or, where none exists yet, its remaining gap written down and signed off by name, plus one page of advice.



Under the hood

- A pinned, checksum-verified secret scan reads every commit of every change, merge commits included, and blocks the merge.
- The rules page is generated from the rule map; CI refuses a page that differs from a fresh render.
- Every pull request waits for an outside AI reviewer, and each point it raises needs a recorded answer before merge.

Results

- Every old rule became a check, signed advice or a decision record, or was deleted.
- A planted violation was refused in CI for **`6 of 8`**[18] rules, client data and secrets among them.

Decisions and trade-offs

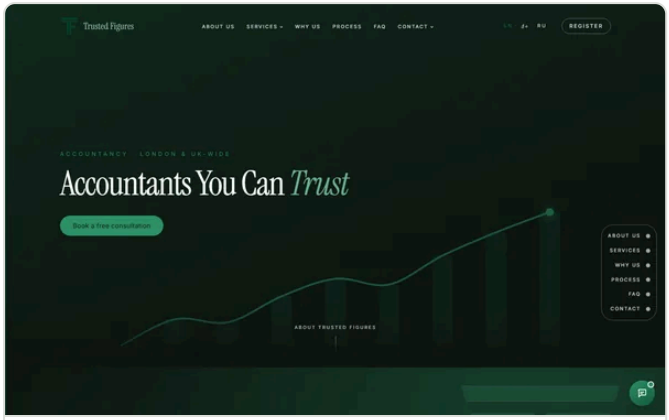
- A gap a check cannot close yet is written down and signed off by name, never covered by a fake check. An ask-first gate for risky commands passed its planted test, then was switched off a day later: it asked so often that I clicked Yes without reading, and a click nobody reads is not a human yes. The gap stays visible instead.
- Checks run on the server, not the laptop. Feedback is slower, but a skipped local check cannot let a change through. When the free CI minutes ran out in October 2026, the written fallback was to merge only with the same checks run locally and pasted into the pull request.

- A check is trusted only after a throwaway pull request plants a real violation and CI refuses it. It costs a pull request per rule, but a check that never fires is found before anyone relies on it.

Skills used in production

Python	`01 CIS pipeline` `03 Bank statements`
TypeScript	`02 Chatbot`
n8n	`04 n8n automations` `02 Chatbot` `01 CIS pipeline`
PostgreSQL / Supabase	`01 CIS pipeline` `02 Chatbot` `04 n8n automations`
LLM extraction with a rules cross-check	`01 CIS pipeline`
AI chatbots with guardrails and a fallback provider	`02 Chatbot`
LLM evaluation with acceptance benchmarks	`02 Chatbot`
Document reading: OCR and PDF text	`01 CIS pipeline` `03 Bank statements`
Financial data in exact decimals	`01 CIS pipeline` `03 Bank statements`
API integrations: WhatsApp Cloud API, Microsoft Graph, Companies House	`02 Chatbot` `04 n8n automations`
Browser tests with Playwright	`01 CIS pipeline`
CI and automated tests: GitHub Actions, pytest, strict type checks	`05 Codebase rules` `01 CIS pipeline` `03 Bank statements` `02 Chatbot`
Secret and personal data scanning	`05 Codebase rules`
AI-assisted development with Claude Code, behind an AI review gate	`05 Codebase rules`
Websites in Next.js and Tailwind, multilingual with next-intl	`Websites`

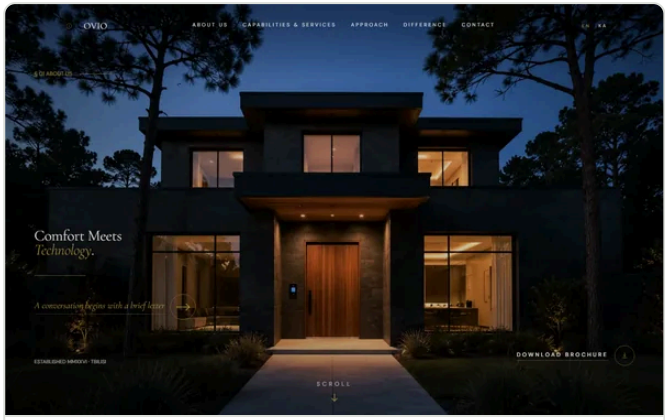
Websites



Trusted Figures

Multilingual site for a London accountancy firm, with online registration and document upload that feeds its automations.

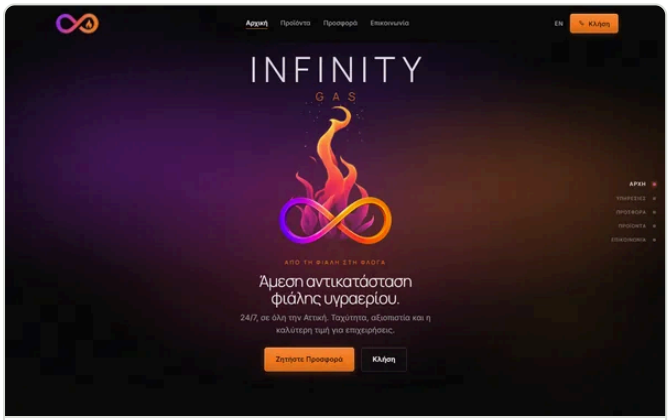
`Next.js, Supabase, Vercel`



OVIO Solutions

Bilingual Georgian and English site for a smart-home studio in Tbilisi, with a contact form that emails the studio.

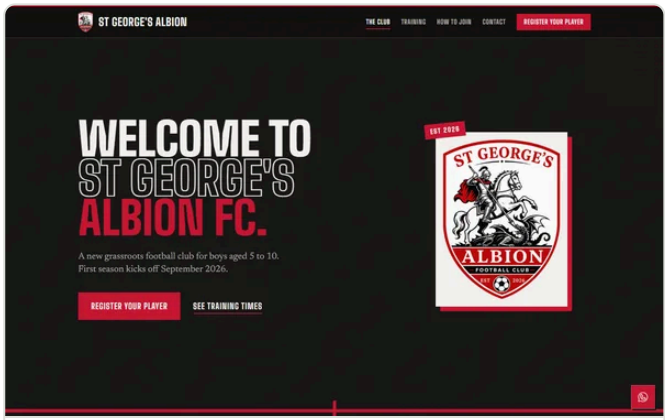
`Next.js, Tailwind, next-intl`



Infinity Gas

Greek and English site for a gas cylinder delivery business in Attica, built around a quote request form.

`Next.js, next-intl, Vitest`



St George's Albion FC

Site for a new youth football club where parents book a free session and register a player online.

`Next.js, Zod, Resend`

How I work

Every system starts with a written scope: what it does, what it must never do, and how we will know it works. Tests come before features. A person approves anything that touches money or a message to a client. Each system ships with handover documents, so someone else can run it.

I build with AI coding tools, under tests and review gates. What I own is the problem framing, the design and its safeguards, checking the results, and running the systems live.

Background

AI and Automation Specialist at Trusted Figures, a London accountancy firm. I started with simple scripts and n8n workflows in 2025. Production systems followed from February 2026, starting with a compliance tracker, and the WhatsApp assistant and the CIS statement pipeline from April 2026.

Languages: English, Georgian, Russian, Greek.

Contact

Write to me by email or on LinkedIn.

ladopaichadze@gmail.com / LinkedIn

How these figures were measured

1. 41: dashboards live on launch day, from the dated project timeline (at launch; measured Aug 2026)
2. 236: read-only count of expected statements, withdrawn ones excluded (38 clients, Q1 2026-27; measured Oct 2026)
3. 42 of 42: acceptance benchmark of 42 defined cases, run against the live AI model; cases passed (live AI model; measured Jul 2026)
4. 140 of 140: every statement's lines re-added to the bank's printed totals and running balance (14,801 transactions, 0 parse failures; measured Jul 2026)
5. 9: count of bank layouts the tool tries, and the brands each one names (11 UK bank brands, 10 tested on real statements; measured Aug 2026)
6. 13: workflows marked active in a read of the live instance (live n8n instance; measured Oct 2026)
7. over 500: 1,053 rule lines from rule files and session prompts, merged into 515 distinct obligations in the review's register (before the review; measured Oct 2026)
8. 8: rows in the must-never table of the generated rules page on main, and items on its advice page (must-never rules, plus one page of advice (40 items); measured Oct 2026)
9. 9: entries in the required-check list the merge ruleset enforces, matching the live ruleset (every merge to main; measured Oct 2026)
10. 3,807: automated tests listed by the test runner on main (the whole CIS pipeline code; measured Oct 2026)
11. 54: read-only count of statements matched to an expected statement (Q1 2026-27; measured Oct 2026)
12. 15: read-only count of exact duplicate files, by fingerprint (since intake went live; measured Oct 2026)
13. 6: count of languages in the assistant's supported list on the main branch (the WhatsApp assistant; measured Oct 2026)
14. 1,315: automated tests run on the main branch with placeholder settings and no live services (the assistant and dashboard code; measured Oct 2026)
15. 8 of 8: photos stored and paired to their messages in a live test; two live tests before the fix stored 5 of 8 (live test after the fix; measured Sep 2026)
16. 976: automated tests listed by the test runner on main (the bank-statement processor's code; measured Oct 2026)
17. 13: active workflows grouped by job from a read of each one's steps: messages in 2, messages out 2, document filing 3, registrations 1, compliance 5 (live n8n instance, by job; measured Oct 2026)
18. 6 of 8: throwaway pull requests that planted a real violation, never merged, whose CI run refused it; each covers the enforced part of its rule (throwaway pull requests, never merged; measured Oct 2026)